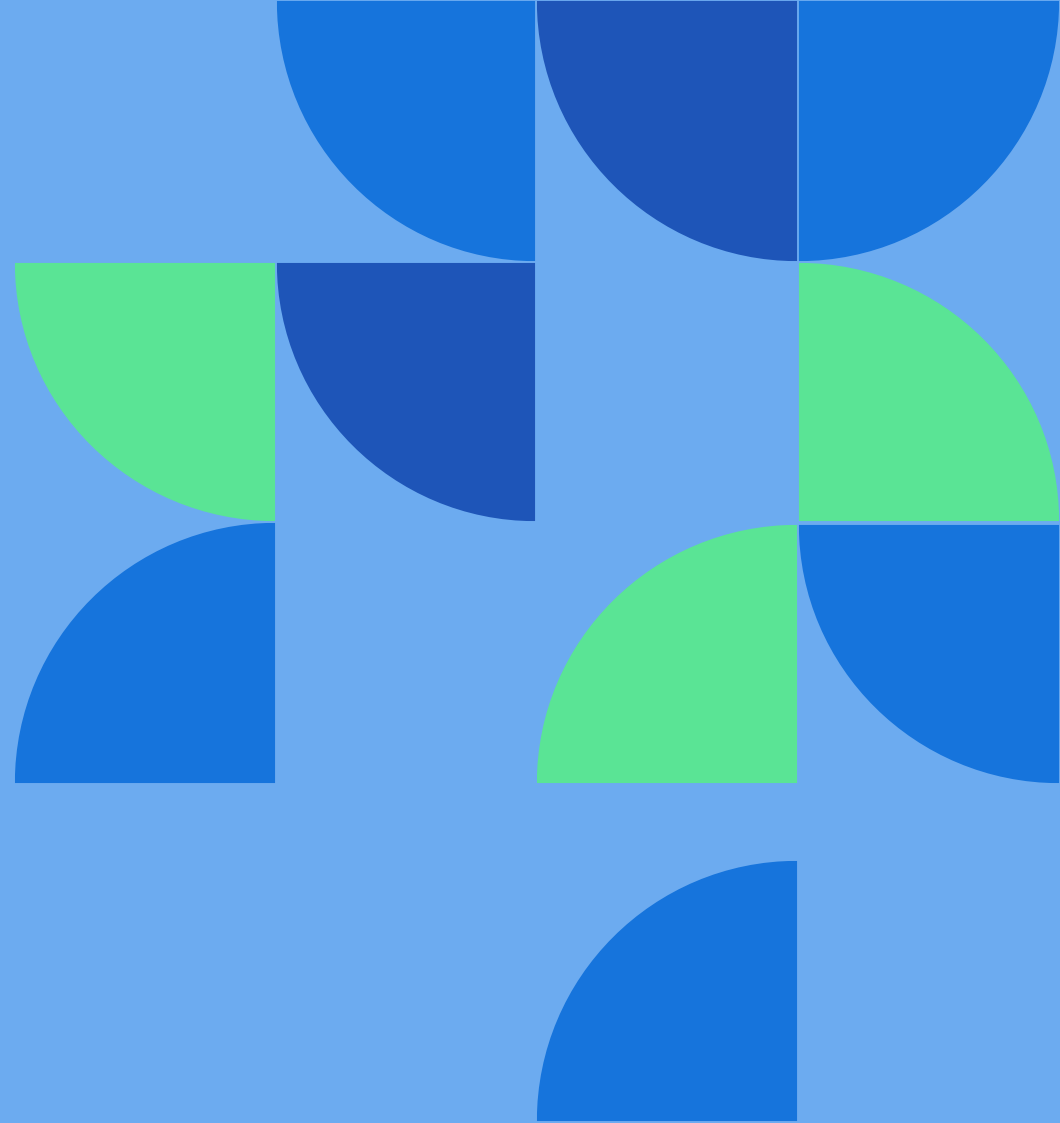


AI-Driven Forecasting and Optimization of Photovoltaic Energy Storage and Market Selling Strategies in European Electricity Markets

Христина Ѓорѓиевска • Огнен Младеновски

Ментор: проф. д-р. Слободан Калајџиски





АГЕНДА

01 ИЗБОР НА
ПОДАТОЧНИ
МНОЖЕСТВА

02 ИСТРАЖУВАЧКА
АНАЛИЗА (EDA)

03 FEATURE
ENGINEERING

04 RQ1 – PRICE
FORECASTING

05 RQ2 – PV PRODUCTION
FORECASTING

06 RQ3 – BATTERY
OPTIMIZATION

07 RQ4 – UNCERTAINTY
ANALYSIS



Наоѓање и избор на податочни множества





ПРЕГЛЕД НА ПОДАТОЧНИТЕ МНОЖЕСТВА

ТИПОВИ НА ПОДАТОЧНИ МНОЖЕСТВА

- Историски пазарни цени на електрична енергија
- Податоци за фотоволтаично производство
- Меторолошки податоци

ДРЖАВИ ОПФАТЕНИ ВО ИСТРАЖУВАЊЕТО

- Германија, Грција, Холандија, Полска, Шпанија и Шведска.

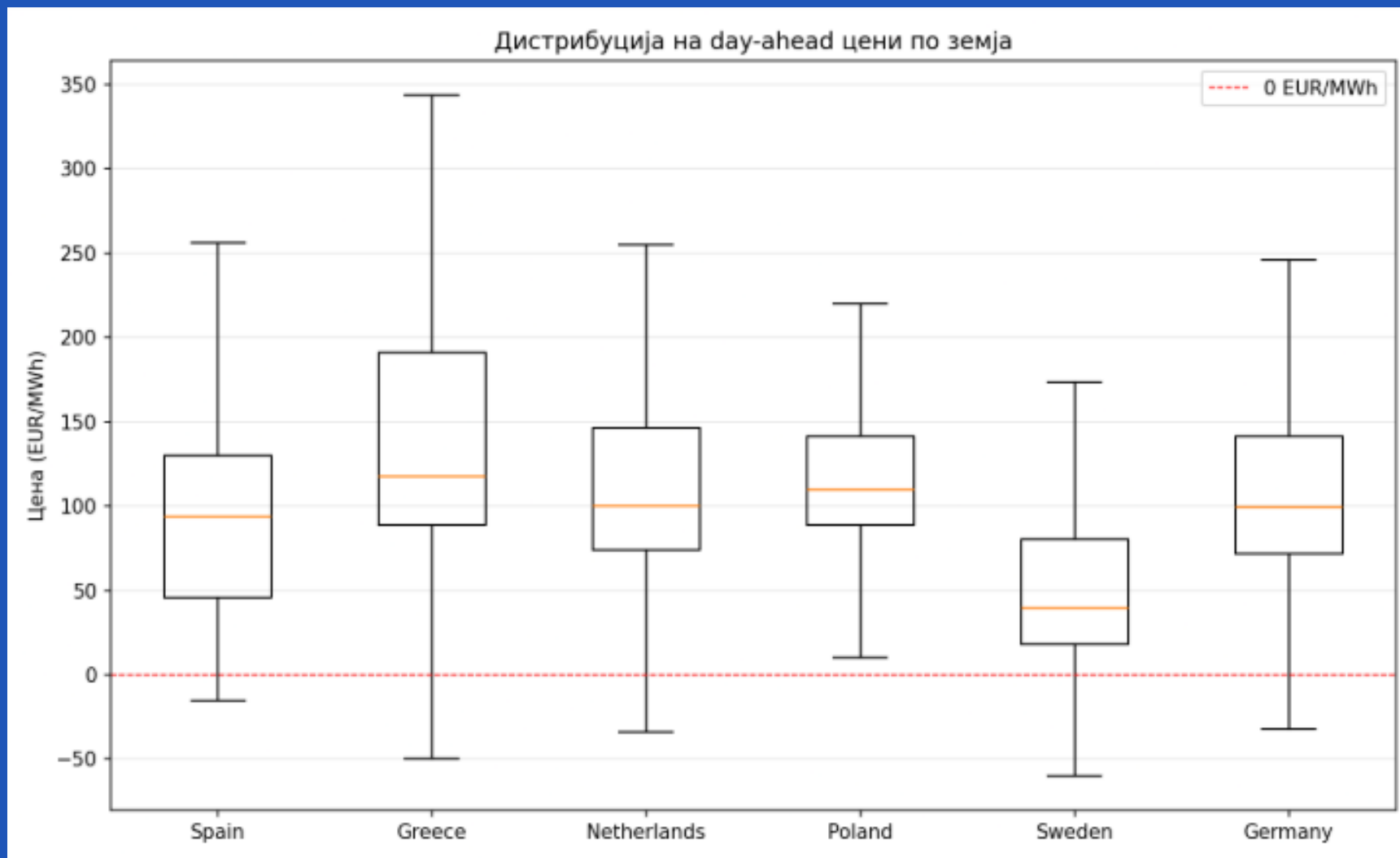
ВРЕМЕНСКИ ОПФАТ

- Секое податочно множество опфаќа период од 4 години на часовно ниво (јануари **2022** – декември **2025**) што резултира со 34 872 часовни записи по земја.
- 

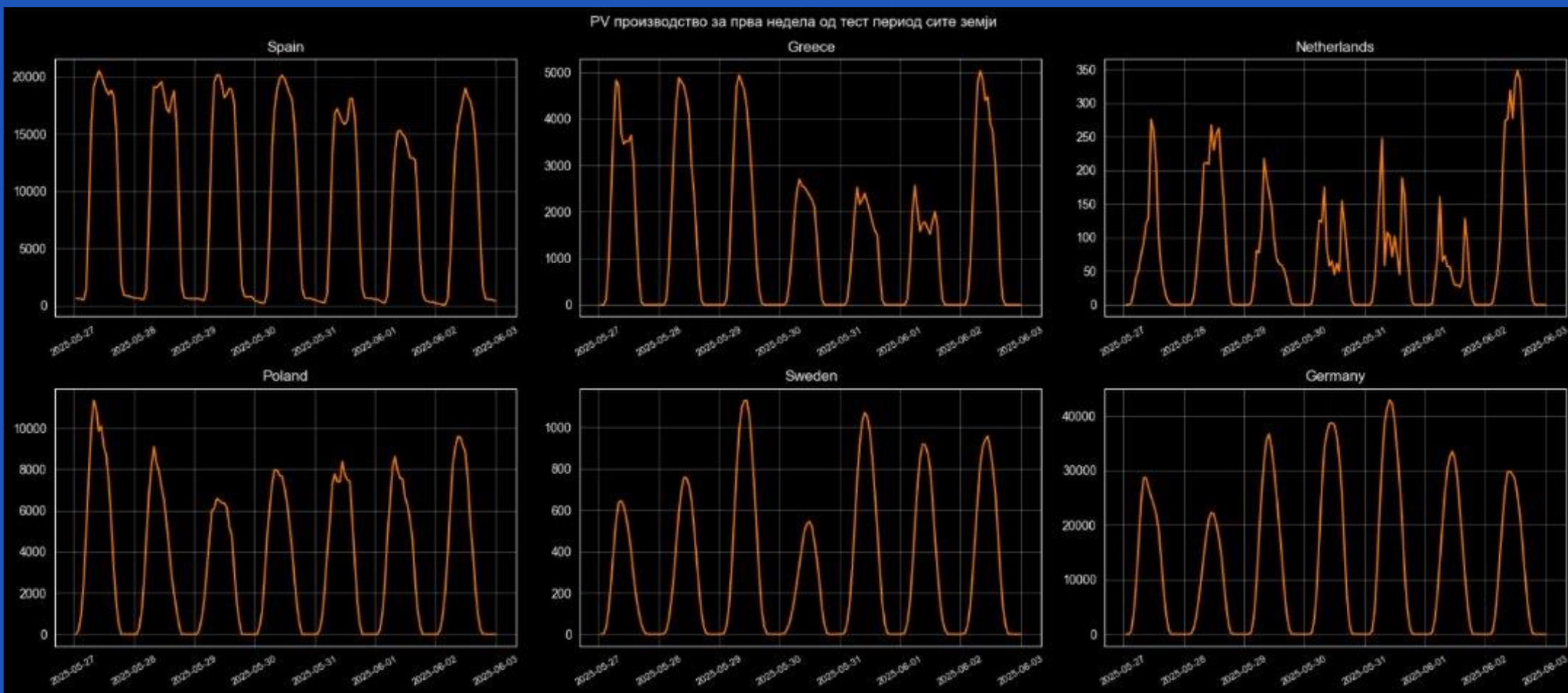
ИСТРАЖУВАЧКА АНАЛИЗА (EDA) (1)

Земја	Просечна цена	Ст. девијација (цена)	% часови со негативна цена	Просечно производство	Максимално производство
Шпанија	95.4	67.2	2.3%	4,907.8	23,999
Грција	150.7	107.4	0.4%	1,080.3	6,816
Холандија	125.3	103.7	4.1%	58.4	427.7
Полска	119.8	63.1	1.6%	1,693.5	13,682.3
Шведска	62.4	73.5	3.8%	104	1,200.3
Германија	124.7	105	3.3%	7,146.7	52,466.2

ИСТРАЖУВАЧКА АНАЛИЗА (EDA) (2)

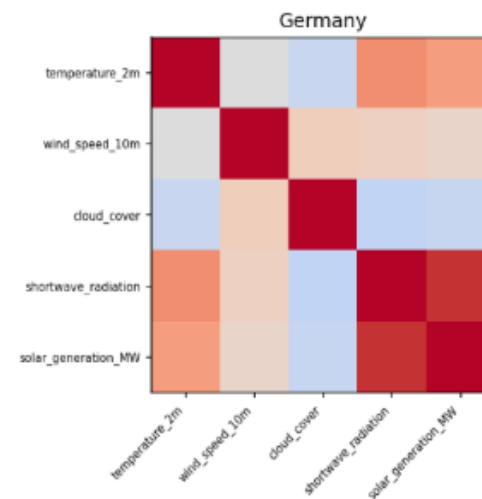
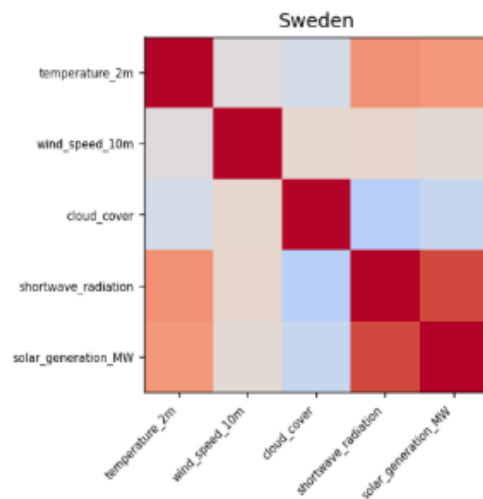
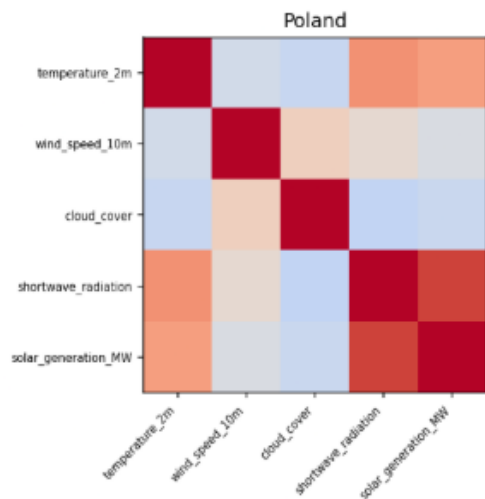
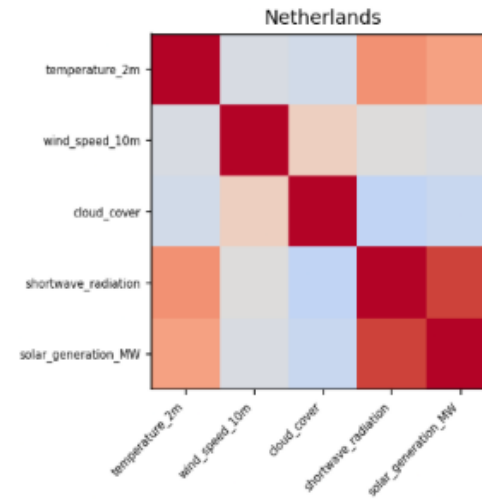
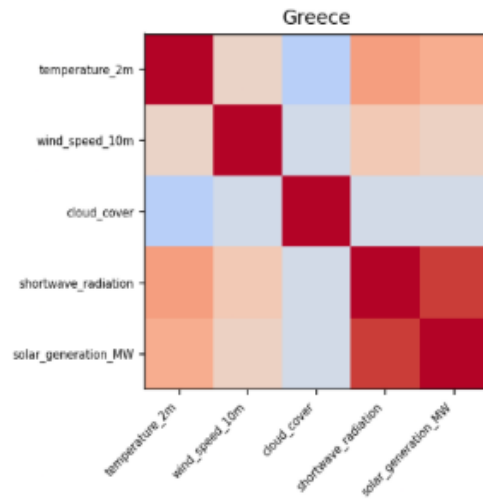
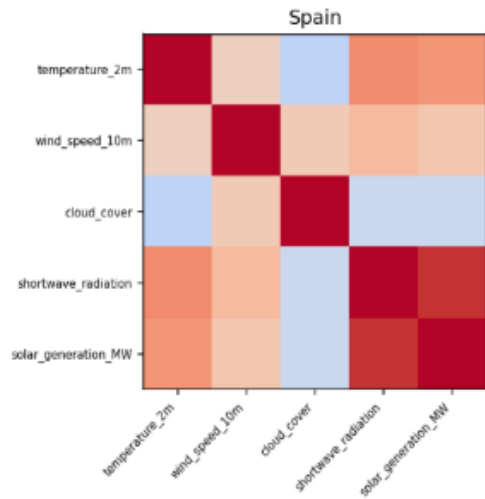


ИСТРАЖУВАЧКА АНАЛИЗА (EDA) (3)



ИСТРАЖУВАЧКА АНАЛИЗА (EDA) (4)

Корелација: временски услови vs PV производство



FEATURE ENGINEERING (1)

- Откако ги споивме трите извори во единствена табела (**datetime + цена + производство + временски услови**), суровите податоци сами по себе не се доволни за да можеме да предвидуваме нешто, затоа од овие сурови податоци изведовме дополнителни карактеристики кои носат временска информација.

1. Lag features

2. Target features

3. Календарски карактеристики



FEATURE ENGINEERING (2)

ХРОНОЛОШКА ПОДЕЛБА

Пред тренирање на било кој модел, целата табела ја делимена **три последователни, непреклопувачки периоди (70%, 15%, 15%)**:

```
11 train_end = int(len(df) * 0.7)
12 val_end = int(len(df) * 0.85)
13 test = df.iloc[val_end:]
```

СКАЛИРАЊЕ НА ПОДАТОЦИ

Невронските мрежи (LSTM/TFT) работат многу подобро кога влезните вредности се во слична скала, за разлика од tree-based моделите (XGBoost) кои не се чувствителни на скалата на податоците.

```
30 scaler_x = StandardScaler().fit(df[features].iloc[:train_end])
31 scaler_y = StandardScaler().fit(df[[target]].iloc[:train_end])
32
33 X_all = scaler_x.transform(df[features])
34 y_all = scaler_y.transform(df[[target]]).ravel()
```

ПРЕВЕНЦИЈА ОД DATA LEAKAGE!!!



RQ1 – Предвидување на цената на електричната енергија



МОДЕЛИ КОИ ГИ ТРЕНИРАМЕ

Baseline Model

79.2%

01

02

XGBoost

88.6%

LSTM

82.1%

03

04

TFT

51.7%





ЗАКЛУЧОЦИ

Кога ги видовме сите резултати заедно, дојдовме до јасен заклучок дека **XGBoost** е секаде најдобар модел во сите земји кои ги тестиравме (Шпанија, Грција, Холандија, Полска, Шведска, Германија). Редоследот е секаде ист: **XGBoost > LSTM > TFT**.

Направивме проверка дали **XGBoost** моделот имал **overfitting**, но резултатите на тренинг и тест сетот беа блиски (разлика од само **4-11** процентни поени по земја), што значи дека моделот навистина научил шаблони, не само специфични примери.

На крај, направивме и дополнително тестирање, да увидиме дали п оголемите пазари (со поголемо **PV** производство) се полесни за предвидување. Резултатот беше негативен, **не постои таква врска**.





RQ2 –Предвидување на фотоволтаично производство

МОДЕЛИ КОИ ГИ ТРЕНИРАМЕ

Baseline Model

89.7%

01

02

XGBoost

94.5%

LSTM

96.1%

03

04

TFT

85.4%

ЗАКЛУЧОЦИ

НАЈДОБАР МОДЕЛ

LSTM е најдобар модел на краток хоризонт ($h=1$) со просек од **96.1%**, потоа по него следи XGBoost (**94.5%**) – за разлика од RQ1. Тука редоследот е **LSTM > XGBoost > TFT**.

ПРЕДВИДЛИВОСТ

PV производството е многу попредвидливо од цени

DISTRIBUTION SHIFT

Distribution shift: доволен временски опфат на податоците (4 години наспроти 3 месеци) целосно го решава distribution-shift проблемот кој го имавме во почетната фаза.

U-ОБЛИК

Baseline моделот покажува екстреман **U-облик по хоризонт** (одличен на $h=1$ и $h=24$, но многу лош во средината – R^2 доаѓа и до – 255% на $h=12$), бидејќи **PV производството оди од нула (ноќе) до максимум (напладне) и назад**



RQ3 - Оптимизација на складирањето на енергија во батерија



КОЈА СТРАТЕГИЈА ЗА СКЛАДИРАЊЕ И ПРОДАЖБА НА ЕНЕРГИЈА ГО МАКСИМИЗИРА ПРИХОДОТ ПРИ НЕИЗВЕСНИ ИДНИ ЦЕНИ И ПРОИЗВОДСТВО?

ПРИСТАП:

Ги користиме предвидувањата од **RQ1** и **RQ2** како информации достапни во моментот на одлучување.

За секој час имаме **24-часовен хоризонт на предвидувања** за цена и производство.

Врз основа на овие информации ги споредуваме 3 стратегии:

- **Rule-based**
- **MPC**
- **Perfect foresight**

ЦЕЛ:

Максимизирање на приходот од користење, складирање и продажба на произведената енергија.



ПОСТАВКИ И ОГРАНИЧУВАЊА НА БАТЕРИЈА

КАПАЦИТЕТ

2 × просечното производство во train периодот

КОГА РАБОТИ БАТЕРИЈАТА?

Батерија работи само меѓу Min SOC=10% и Max SOC=90%

БРЗИНА НА ПОЛНЕЊЕ/ПРАЗНЕЊЕ

40% од капацитетот на час

ЕФИКАСНОСТ НА ПОЛНЕЊЕ/ПРАЗНЕЊЕ

95%, бидејќи секое полнење/празнење губи мал дел енергија како топлина

RULE-BASED МОДЕЛ

За секој час, гледа дали тековната (познатата) цена е над однапред пресметан праг (75-ти перцентил од **train**), ако е тогаш продава/празни батерија, а ако е под 25-ти перцентил, полни батерија.

```
21 low_price = train['Price (EUR/MWhe)'].quantile(0.25)
22 high_price = train['Price (EUR/MWhe)'].quantile(0.75)
23
24 soc = MIN_SOC
25 revenue = 0
26
27 for i, row in df.iterrows():
28     production = row['actual_P']
29     real_price = row['actual_price']
30
31     charge = 0
32     discharge = 0
33     sell = production
34
35     if real_price > high_price:
36         discharge = min(MAX_RATE, soc - MIN_SOC)
37     elif real_price < low_price:
38         charge = min(production, MAX_RATE, MAX_SOC - soc)
39         sell = production - charge
40
41     revenue += real_price * (sell + discharge * EFFICIENCY) / 1000
42     soc = soc + charge * EFFICIENCY - discharge
```

MPC (MODEL PREDICTIVE CONTROL)

Математичка техника која наоѓа најдобра комбинација на броеви (колку да се продаде, колку да се наполни, колку да се испразни батеријата секој час) која го максимизира вкупниот приход, притоа да почитува точно дефинирани ограничувања.

За секој 24-часовен прозорец, солверот (PuLP/CBC) бара вредности за секој час t кои:

- (1) го максимизираат вкупниот приход (цена $\times$ продадена + испразнета енергија),
- (2) обезбедуваат дека целото произведено количество некаде оди (продажба/полнење),
- (3) го следат точното менување на состојбата на батеријата час по час
- (4) не дозволуваат батеријата да прелее преку максимумот или да падне под минимумот, ниту да се полни/празни побрзо од дозволената брзина

```
10 def solve_window(prices, productions, soc, max_soc, min_soc, max_rate): 1 usage & hristinagjorgjievska *
11     H = len(prices)
12     prob = pulp.LpProblem( name: "mpc", pulp.LpMaximize)
13
14     sell = [pulp.LpVariable( name: f"sell_{t}", lowBound=0) for t in range(H)]
15     charge = [pulp.LpVariable( name: f"charge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
16     discharge = [pulp.LpVariable( name: f"discharge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
17     battery = [pulp.LpVariable( name: f"soc_{t}", lowBound=min_soc, upBound=max_soc) for t in range(H)]
18
19     prob += pulp.lpSum(prices[t] * (sell[t] + discharge[t] * EFFICIENCY) for t in range(H))
20
21     for t in range(H):
22         prob += sell[t] + charge[t] == productions[t]
23         prev_soc = soc if t == 0 else battery[t - 1]
24         prob += battery[t] == prev_soc + charge[t] * EFFICIENCY - discharge[t]
25
26     prob.solve(pulp.PULP_CBC_CMD(msg=0))
27     return sell[0].varValue, charge[0].varValue, discharge[0].varValue
28
```

```
50     for t in range(n):
51         row = df.iloc[t]
52         remaining = min(HORIZON, n - t)
53
54         future_prices = row[price_cols].values[:remaining - 1]
55         future_prod = row[pv_cols].values[:remaining - 1]
56
57         window_prices = [row['actual_price']] + list(future_prices)
58         window_prod = [row['actual_P']] + list(future_prod)
59
60         sell, charge, discharge = solve_window(
61             window_prices, window_prod, soc, MAX_SOC, MIN_SOC, MAX_RATE
62         )
63
64         real_price = df['actual_price'].iloc[t]
65         real_production = df['actual_P'].iloc[t]
66
67         actual_sell = min(sell, real_production)
68         actual_charge = min(charge, real_production - actual_sell, MAX_SOC - soc)
69         actual_discharge = min(discharge, soc - MIN_SOC)
70
71         surplus = real_production - actual_sell - actual_charge
72         actual_sell += surplus
73
74         revenue += real_price * (actual_sell + actual_discharge * EFFICIENCY) / 1000
75         soc = soc + actual_charge * EFFICIENCY - actual_discharge
```

PERFECT-FORESIGHT

Perfect Foresight ја користи истата LP логика како **MPC**, со единствена разлика, наместо прогнозирани идни вредности за цена и производство, во прозорецот се внесуваат вистинските, реални идни вредности (кои во реалноста не би биле познати однапред).

```
10 def solve_window(prices, productions, soc, max_soc, min_soc, max_rate): 1 usage 2 hristinagjorgjievska *
11 H = len(prices)
12 prob = pulp.LpProblem(name="mpc", pulp.LpMaximize)
13
14 sell = [pulp.LpVariable(name=f"sell_{t}", lowBound=0) for t in range(H)]
15 charge = [pulp.LpVariable(name=f"charge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
16 discharge = [pulp.LpVariable(name=f"discharge_{t}", lowBound=0, upBound=max_rate) for t in range(H)]
17 battery = [pulp.LpVariable(name=f"soc_{t}", lowBound=min_soc, upBound=max_soc) for t in range(H)]
18
19 prob += pulp.lpSum(prices[t] * (sell[t] + discharge[t] * EFFICIENCY) for t in range(H))
20
21 for t in range(H):
22     prob += sell[t] + charge[t] == productions[t]
23     prev_soc = soc if t == 0 else battery[t - 1]
24     prob += battery[t] == prev_soc + charge[t] * EFFICIENCY - discharge[t]
25
26 prob.solve(pulp.PULP_CBC_CMD(msg=0))
27 return sell[0].varValue, charge[0].varValue, discharge[0].varValue
28
```

```
45 for t in range(n):
46     window = df.iloc[t:t+HORIZON]
47     sell, charge, discharge = solve_window(
48         window['actual_price'].values, window['actual_P'].values, soc, MAX_SOC, MIN_SOC, MAX_RATE
49     )
50
51     real_price = df['actual_price'].iloc[t]
52     real_production = df['actual_P'].iloc[t]
53
54     actual_sell = min(sell, real_production)
55     actual_charge = min(charge, real_production - actual_sell, MAX_SOC - soc)
56     actual_discharge = min(discharge, soc - MIN_SOC)
57
58     surplus = real_production - actual_sell - actual_charge
59     actual_sell += surplus
60
61     revenue += real_price * (actual_sell + actual_discharge * EFFICIENCY) / 1000
62     soc = soc + actual_charge * EFFICIENCY - actual_discharge
```

РЕЗУЛТАТИ

Земја	Rule-based	MPC	Perfect Foresight
Шпанија	1.348.588 EUR	1.454.609 EUR	1.465.655 EUR
Грција	392.580 EUR	423.891 EUR	428.938 EUR
Холандија	16.519 EUR	17.886 EUR	18.079 EUR
Полска	838.562 EUR	863.483 EUR	873.354 EUR
Шведска	23.429 EUR	24.139 EUR	24.646 EUR
Германија	2.134.283 EUR	2.285.337 EUR	2.311.480 EUR

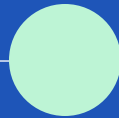
ЗАКЛУЧОЦИ

ПЕРФОРМАНС НА MPC



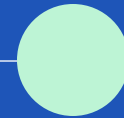
MPC е подобар од **rule-based** за секоја земја, со што се потврдува дека формалната оптимизација носи реална вредност над едноставните (rule-based) прагови

ОПТИМАЛНОСТ



MPC постигнува многу близу до теоретски оптимално, дури и со реалистични, несовршени прогнози, се губи само **1-2%** од максималниот можен приход

СЛОЕВИТА АРХИТЕКТУРА



RQ3 директно ги презема веќе тестираните и потврдени предвидувања од **RQ1** и **RQ2**

RQ4 – Влијание на неизвесноста врз стратегијата

Колку е чувствителна MPC стратегијата на грешки во предвидувањето и дали грешките во некои услови (облачно, викенд, зима) чинат повеќе отколку во други?



ПРВ ТЕСТ: MONTE-CARLO СИМУЛАЦИЈА

Monte Carlo симулација: генериравме случаен шум (**+–20%** стандардна девијација) околу прогнозите за цена и за производство, одделно, 10 повторувања за секоја и мериме колку паѓа приходот споредено со **base MPC** приходот без шум.

Земја	Загуба - несигурност во цена	Загуба - несигурност во производство
Шпанија	1.91%	~0%
Грција	1.8%	~0%
Холандија	1.76%	~0%
Полска	1.56%	~0%
Шведска	1%	~0%
Германија	1.61%	~0%

ЗАКЛУЧОК:

Несигурноста во **PV** производството е практично безначајна за приходот, додека несигурноста во цената дава конзистентна загуба (во просек околу 1.61%).

Ова значи дека инвестирањето во подобрувањето на предвидувањето на цени (**RQ1**) би донело поголема практична вредност отколку подобрување во предвидувањето на **PV** производството (**RQ2**)

ВТОР ТЕСТ: FIXED PERTURBATIONS

Систематски (**fixed**) нарушувања: наместо да додаваме случаен шум, систематски ги скалираме идните предвидени цени и со фиксен процент (**+10%, +20%, +30%**) и мериме како ќе реагира приходот. Резултатот покажа hump облик, негативно скалирање (ги потценува идните цени), секогаш дава повисок приход од позитивното скалирање (преценување) со иста големина.

Земја	-30%	-20%	-10%	+10%	+20%	+30%
Шпанија	1,404,872	1,427,217	1,447,536	1,436,618	1,386,089	1,356,260
Грција	407,039	418,683	423,920	419,381	403,168	392,507
Холандија	17,143	17,608	17,842	17,687	17,302	16,872
Полска	830,841	845,017	859,103	854,538	837,456	824,360
Шведска	23,572	23,741	23,985	23,982	23,576	23,334
Германија	2,198,609	2,249,049	2,281,165	2,265,096	2,216,204	2,157,183

ЗАКЛУЧОК:

Објаснување: кога МРС потценува идни цени, тој е претпазлив и продава веднаш по позната, сигурна тековна цена, а кога преценува, тој складира енергија со тоа што очекува повисоки идни цени кои во реалноста не би се материјализирале толку силно.

ТРЕТ ТЕСТ: АНАЛИЗА ПО РЕЖИМИ

Анализа по режими: го поделивме тест периодот на парови од спротивни режими (висока/ниска волатилност, облачно/сончево, високо/ниско производство, викенд/работен ден, price spike/no spike, зима/лето) и мериме % загуба на приход одделно за секој пар.

Земја	High vol.	Low vol.	Cloudy	Sunny	High PV	Low PV	Weekend	Weekday	Price spike	No spike	Winter	Summer
Шпанија	2.64%	1.11%	1.75%	1.94%	1.78%	1.45%	1.78%	1.81%	1.77%	1.98%	1.51%	1.80%
Грција	2.47%	1.02%	2.35%	1.41%	1.27%	2.49%	1.64%	1.89%	1.97%	1.35%	1.77%	1.28%
Холандија	2.67%	1.15%	2.76%	1.43%	1.15%	3.66%	1.29%	1.83%	1.83%	1.23%	3.32%	1.37%
Полска	1.79%	1.21%	1.98%	1.30%	1.22%	2.18%	1.58%	1.32%	1.53%	1.45%	2.40%	1.27%
Шведска	1.48%	0.49%	1.41%	0.83%	0.47%	2.35%	0.63%	1.01%	1.15%	0.84%	1.58%	0.67%
Германија	2.48%	1.09%	2.02%	1.63%	1.71%	2.25%	1.00%	1.72%	1.84%	1.15%	1.45%	1.28%

ЗАКЛУЧОЦИ

- **Високата волатилност** (кога цената многу се менува) е најголемиот ризик. Ова важи за сите 6 земји, загубата е **2,6 до 5,4 пати поголема** отколку кога цената е стабилна. Значи, кога цените многу се менуваат, грешките во предвидувањето имаат многу поголемо влијание.
- **Холандија е најслаба во речиси сите случаи**, особено во зима (**3,32% загуба**, наспроти **1,37% во лето**) и во денови со малку сонце (**3,66%**, наспроти **1,15% кога има многу сонце**). Ова покажува дека нејзиниот помал и понестабилен PV капацитет не само што е потежок за предвидување (како во RQ1/RQ2), туку и **грешките таму имаат поголеми последици**.
- **Шведска е најотпорна земја**, во сите услови има најниски загуби (**0,47%–2,35%**). Веројатно причината е тоа што има постабилен пазар со помалку нагли промени на цените.
- **Малото производство на сончева енергија е ризик во повеќето земји (5 од 6)**. Кога има малку сонце, батеријата има помалку енергија со која може да работи, па **секоја грешка во предвидувањето има поголемо влијание**.
- **Зимските денови се поризични од летните во 5 од 6 земји**, веројатно затоа што во зима производството е помало и понестабилно, а цените почесто растат поради поголемата побарувачка за греење.
- **Препорака:** ако сакаме стратегијата да биде **постабилна и помалку ризична**, најмногу треба да се подберат прогнозите за **нестабилни цени, зимски периоди и денови со ниско производство**, токму таму грешките во моментот чинат најмногу.